Presenter Guide: Progress Report (Group 4)

Oct 30, 2026 · @Raj Panchal

The project in one minute

We want a computer to guess which Python methods developers will **rename** (for example `get_usr` → `get_user`), using only the code **before** the change.

1. We downloaded **ActRef**, real code from 1,160 changes in Python projects. It had the code but **no answer key**.
2. So we built our own: a script found **430 renames** (the ActRef paper says 432, so it works). After cleaning and review we kept **243**.
3. We added methods that were **not** renamed, turned every method into numbers (clues), and trained models.
4. First result looked great (2.5× better than guessing). We checked it and found it came from **how we built the data**, not a real pattern.
5. After fixing it, the honest result: models are only about **2× better than guessing**. Renaming is hard to predict from code alone.
6. Next: try **detection** (look at the code before *and* after), test on **PyRef**, write the report.

If you remember one sentence: *we built our own labelled dataset, trained models, caught a mistake in our own evaluation, and found that predicting renames from code alone is hard.*

Key words

Word	Simple meaning
Method	A small named piece of code that does one job, e.g. `def get_user():`
Rename Method refactoring	Changing a method's name without changing what it does
Commit	One saved change to a project's code
Before / After	The code just before and just after a commit

Word	Simple meaning
ActRef	2025 research dataset of real Python code changes (our training data)
PyRef	Older dataset with 139 known renames (our test data, used later)
Label	The answer for one example: renamed (1) or not renamed (0)
Positive / Negative	A renamed method / a method that was not renamed
Feature	One measured clue about a method, e.g. name length or lines of code
Baseline	A very simple guesser we must beat, e.g. always say "not renamed"
Cross-validation	Split data into 5 parts; train on 4, test on 1; repeat 5 times
PR-AUC	Our main score: how well renamed methods are ranked on top. Random guessing scores the share of renamed methods
Ablation	Remove a group of features and retrain, to see how much that group really helps

Who presents what

A suggested split, about 5 minutes each. Change it to suit you.

Part	Slides	Presenter	Topic
1	1–4	Raj	Question, progress, pipeline
2	5–10	Raj	Steps 1–6: getting and labelling the data
3	11–13	Johnson	Steps 7–9: negatives, features, training
4	14–19	Johnson	Results, ablation check, lessons, next steps

Handover line (Raj, end of slide 10): *"Now we have 243 renamed methods. Johnson will explain how we added the not-renamed ones and trained the models."*

Script: slides 1–4 (opening)

Slide 1 · Cover

> "Hello, we are Group 4. Our project asks whether machine learning can predict when a Python method will be renamed. Today we show what we built, our first results, and a mistake we found and fixed in our own evaluation."

Slide 2 · The question

> "We look only at a method's code *before* a commit and ask: will it be renamed? We have three questions: does ML beat simple guessing, which clues matter most, and does it work on another dataset, PyRef."

Slide 3 · Progress

> "All nine steps from downloading the data to training models are done. Still to do: a detection experiment, the PyRef test, explaining the model, and the report."

Slide 4 · Pipeline

> "Here is the whole process in five boxes: take the code, detect renames, review them, turn each method into numbers, train models. Everything the model sees comes from the *Before* code, so it can't cheat by seeing the new name."

Script: slides 5–13 (the nine steps)

Every step slide has three boxes: **What we did**, **Why**, **Result**. Read them left to right.

Slide 5 · Step 1, Download

> "We downloaded ActRef, a 2025 dataset of real Python code changes. We got 1,160 commits. We needed lots of real examples of renamed methods to teach a model."

Slide 6 · Step 2, Explore

> "Each commit has a Before and an After folder of code. But there was no file saying which methods were renamed. No answer key, so we had to build one."

Slide 7 · Step 3, Detect

> "Our script compares Before and After. If a method disappears and a new name appears with almost the same code inside, that is a rename. It found 430. The ActRef paper reports 432, so our detector works."

Slide 8 · Step 4, Clean

> "We removed 96 test methods, because they are renamed for different reasons. We split the rest into confident cases, with nearly identical code, and unsure cases."

Slide 9 · Step 5, Review

> "We checked the 149 unsure cases one by one with a simple rule: a rename must keep the same job. `download_url` to `download_urls` is a yes. `user_store` to `sets_store` is a no, two different methods. We kept 118."

Slide 10 · Step 6, Final labels

> "One commit renamed 48 methods at once, so we allowed at most 5 per commit to stop it dominating. Final result: 243 renamed methods."

Handover to Johnson.

Slide 11 · Step 7, Negatives

> "To learn what 'renamed' looks like, the model must also see 'not renamed'. These are methods that kept their name. First we took 1,915 from the same files; later 30,974 from all commits."

Slide 12 · Step 8, Features

> "Each method becomes a list of numbers in four groups: the name, the code, the context around it, and how it compares with its neighbours. We never use the new name or the commit message, because that gives away the answer."

Slide 13 · Step 9, Train

> "We trained three simple baselines and three ML models. We test with 5-fold cross-validation, keeping each project on one side so the model can't memorise a project. Our main score is PR-AUC. We don't use accuracy: always saying 'no' is already 99% accurate."

Script: slides 14–19 (results and next steps)

Slide 14 · Experiment 1, bar chart

> "Grey bars are simple guessers, around 0.11. Our best model, Random Forest, reached 0.29, about 2.5 times better. It looked like a success, but the top clue was file size, which seemed strange. So we checked."

Slide 15 · Ablation check (most important slide)

> "We removed the file-size clues and the score fell to 0.14, almost random. The reason: we only took not-renamed methods from files that had a rename. In a tiny file, one rename is a big share, so the model learned 'small file means rename'. That came from how we built the data, not from real developer behaviour."

Analogy if needed: *a student who passes only because every answer on the test was 'A'.*

Slide 16 · Experiment 2, the fix

> "We fixed it by taking not-renamed methods from all commits: 31,217 methods, only 0.8% renamed, which is realistic. Random guessing now scores 0.008. Our best models reach about 0.02, about twice as good. A real signal, but weak."

Slide 17 · What we learned

> "Three lessons. One: ML beats guessing, but only a little. Two: obvious 'bad name' clues barely help; the file and class context help most. Three: how you build the data can fake a good result, and checking caught it. Our explanation: developers rename for reasons the code doesn't show, like an API redesign or a team naming rule."

Slide 18 · Next steps

> "Next week we add detection: look at the code before and after, and decide whether a change was a rename. Then we test on PyRef, explain which clues matter with SHAP, and write the final report."

Slide 19 · Thank you

> "Thank you. We're happy to take questions."

Likely questions and simple answers

Question	Answer
Why are your scores so low?	Because the task is genuinely hard: one snapshot of code doesn't show *why* developers rename. That is a finding in itself.
Can we trust your labels?	Our detector found 430 renames; the ActRef paper reports 432. Unsure cases were reviewed one by one.
Why didn't you use ActRef's own labels?	The public download contains only the code, not the label file.
What is PR-AUC?	A score for how well the model ranks renamed methods above the others. Random guessing scores the share of renamed methods (0.008 on our realistic data).
Why not accuracy?	Fewer than 1% of methods are renamed, so always saying "no" is 99% accurate and useless.

Question	Answer
Why remove test methods?	Tests are renamed for different reasons (e.g. describing a new test case) and would confuse the model.
Why cap 5 renames per commit?	One commit renamed 48 methods at once; without a cap it would dominate training.
Why group cross-validation by project?	So the model can't score well just by recognising a project it has already seen.
What is the difference between prediction and detection?	Prediction: guess *before* the change. Detection: look at before *and* after and decide if it was a rename. Detection is much easier.
When do you test on PyRef?	Week of November 9, after removing any commits PyRef shares with ActRef.

If you don't know an answer: *"Good question, we'll look into that for the final report."*